

MITgcm KPP Package

K-Profile Parameterization for Vertical Ocean Mixing

Technical Reference: Architecture, Physics, and Implementation

Package Location: MITgcm/pkg/kpp/

Primary Reference: Large, W.G., McWilliams, J.C., Doney, S.C. (1994). *Oceanic vertical mixing: a review and a model with a nonlocal boundary layer parameterization*. Reviews of Geophysics, 32(4), 363–403.

Additional Reference: van Roekel et al. (2018). *The KPP Boundary Layer Scheme for the Ocean: Revisiting Its Formulation and Benchmarking One-Dimensional Simulations Relative to LES*. JAMES, 10, 2647–2685.

Compiled from source files:

KPP.h	State variable declarations
KPP_OPTIONS.h	Compile-time options
KPP_PARAMS.h	Run-time parameters
kpp_calc.F	Main driver routine
kpp_routines.F	Core physics subroutines
kpp_forcing_surf.F	Surface forcing preparation
kpp_calc_diff_t.F	Temperature diffusivity interface
kpp_calc_diff_s.F	Salinity diffusivity interface
kpp_calc_diff_ptr.F	Passive tracer diffusivity interface
kpp_calc_visc.F	Viscosity interface
kpp_transport_t.F	Non-local temperature flux
kpp_transport_s.F	Non-local salinity flux
kpp_transport_ptr.F	Non-local passive tracer flux
kpp_init_fixed.F	Time-invariant initialization
kpp_init_varia.F	Time-varying initialization
kpp_readparms.F	Parameter file reader
kpp_output.F	Diagnostics and I/O
kpp_check.F	Consistency checks
kpp_do_exch.F	Halo exchanges
kpp_diagnostics_init.F	Diagnostics registration

August 12, 2026

Contents

1	Introduction and Scientific Background	4
1.1	Physical Motivation	4
1.2	Two-Region Decomposition	4
1.3	Key State Variables	5
2	Governing Equations	5
3	Package Architecture and Call Flow	7
3.1	Initialization Sequence	7
3.2	Per-Timestep Execution Flow	7
3.3	Execution Frequency	8
4	Parameter Initialization	9
4.1	KPP_READPARMS – Reading <code>data.kpp</code>	9
4.2	KPP_INIT_FIXED – Time-Invariant Initialization	10
4.2.1	Construction of Turbulent Velocity Scale Lookup Tables	10
4.2.2	Vertical Grid Setup	11
4.3	KPP_INIT_VARIA – Variable Initialization	11
5	Main Driver: KPP_CALC	12
5.1	Velocity Shear Computation	12
5.2	Background Diffusivity and Double Diffusion	13
5.3	Result Transfer and Masking	14
6	Stratification and Buoyancy: STATEKPP	14
6.1	Outputs	14
6.2	Key Density Calls	15
7	Surface Forcing: KPP_FORCING_SURF	16
7.1	Friction Velocity	16
7.2	Surface Buoyancy Fluxes	16
7.3	Surface-Referenced Velocity Shear <code>dVsq</code>	17
8	Interior Mixing: Ri_iwmix	17
8.1	Richardson Number Computation	17
8.2	Mixing Functions	18
8.3	Vertical Smoothing of Ri (<code>z121</code>)	19
9	Boundary / Surface Layer Mixing	19
9.1	Boundary Layer Depth Diagnosis: <code>bldepth</code>	20
9.2	Bulk Richardson Number	20
9.3	Linear Interpolation for <code>hbl</code>	20
9.4	Stability Limits on <code>hbl</code>	21

9.5	Case A / Case B Classification	21
9.6	Turbulent Velocity Scales: <code>wscale</code>	22
9.7	Bilinear Interpolation	22
9.8	Boundary Layer Mixing Coefficients: <code>blmix</code>	22
9.9	Shape Function $G(\sigma)$	22
9.10	Non-local Transport Term	24
9.11	Interface Enhancement: <code>enhance</code>	24
10	Core Diffusivity and Prognostic Update: KPPMIX	25
11	Additional Mixing Processes	25
11.1	Double Diffusion: <code>KPP_DOUBLEDIFF</code>	26
11.2	Salt Fingering ($\alpha dT > \beta dS > 0$)	26
11.3	Diffusive Convection ($\alpha dT < 0, \beta dS < 0$)	26
11.4	Shortwave and Salt Plume Fraction Diagnostics	27
11.5	<code>KPPfrac</code> – Shortwave Penetration Fraction	27
11.6	<code>KPPplumefrac</code> – Salt Plume Penetration Fraction	27
12	Eddy Coefficients and Model Interface	27
12.1	<code>KPP_CALC_DIFF_T</code> and <code>KPP_CALC_DIFF_S</code>	27
12.2	<code>KPP_CALC_DIFF_PTR</code>	28
12.3	<code>KPP_CALC_VISC</code>	28
12.4	<code>KPP_TRANSPORT_T</code>	28
12.5	<code>KPP_TRANSPORT_S</code>	29
13	Initialization and Restart	30
13.1	KPP is Diagnostic, Not Prognostic	30
13.1.1	What “Diagnostic” Means	30
13.1.2	No State Persistence Between Time Steps	30
13.1.3	No Pickup/Restart Files for KPP	30
13.2	Initialization	31
13.3	Implications of Diagnostic Formulation	31
13.3.1	Advantages	31
13.3.2	Limitations	31
13.4	Contrast with GGL90 (Prognostic TKE Scheme)	32
13.5	Practical Restart Workflow	32
13.6	Recommendations	32
14	Compile-Time Options	33
15	Output and Diagnostics	34
15.1	State File Output (<code>KPP_OUTPUT</code>)	34
15.2	Online Diagnostics (<code>KPP_DIAGNOSTICS_INIT</code>)	34
16	Package Validation: <code>KPP_CHECK</code>	35

17 Numerical Considerations & Frequently Encountered Issues	35
18 Conclusions	36
19 Glossary of Symbols	37
A Complete Input/Output Mapping	37
A.1 Package-Level Inputs	37
A.2 Package-Level Outputs	38
B Adjoint Model Considerations	39
B.1 Overview: KPP in Adjoint Mode	39
B.1.1 KPP's Adjoint Characteristics	39
B.2 Adjoint-Specific Implementation	40
B.2.1 TAF Directives in KPP Source Code	40
B.2.2 Adjoint Differentiation List	41
B.3 Adjoint-Specific CPP Flags	41
B.3.1 KPP_AUTODIFF_EXCESSIVE_STORE	41
B.4 Known Adjoint Issues and Workarounds	42
B.4.1 Issue 1: Gradient Noise at Boundary Layer Depth Transitions	42
B.4.2 Issue 2: Conditional Branch Discontinuities	43
B.4.3 Issue 3: Lookup Table Interpolation Artifacts	43
B.5 KPP vs. GGL90: Adjoint Complexity Comparison	43
B.6 Best Practices for Adjoint Runs with KPP	44
B.7 Adjoint Validation Test Cases	45
B.8 References for Adjoint Modeling	45

1 Introduction and Scientific Background

The K-Profile Parameterization (KPP) is a first-order turbulence closure scheme designed to represent vertical mixing throughout the full ocean water column. It was developed by Large, McWilliams, and Doney (1994; hereafter LMD94) and has become one of the most widely used vertical mixing parameterizations in ocean general circulation models. The MITgcm implementation closely follows LMD94 but has been adapted for the finite-volume, staggered-grid architecture of MITgcm.

1.1 Physical Motivation

Vertical mixing in the ocean is driven by several distinct physical mechanisms that operate in different parts of the water column:

1. **Surface boundary layer turbulence** – Wind stress and surface buoyancy fluxes drive turbulent kinetic energy into the ocean surface layer. This energy is distributed throughout the oceanic planetary boundary layer (OBL) to depth h_{bl} .
2. **Shear instability** – Vertical shear in the horizontal velocity field can generate turbulence when the local Richardson number falls below a critical threshold.
3. **Static instability / convection** – Unstable density stratification (dense water over light water) drives rapid convective overturning.
4. **Internal wave breaking** – Background internal wave activity maintains a minimum level of mixing throughout the interior ocean.
5. **Double diffusion** – The differential molecular diffusivities of heat and salt can give rise to salt fingering and diffusive convection when the density ratio R_ρ is near unity.

1.2 Two-Region Decomposition

KPP divides the water column into two regions, each with its own mixing parameterization:

Region 1 – Oceanic Boundary Layer ($z > -h_{bl}$):

Mixing is parameterized using a smooth turbulent velocity scale profile that depends on surface forcing, boundary layer depth, and stability. The profile is constrained to match interior mixing at $z = -h_{bl}$. A non-local (counter-gradient) transport term $\hat{\gamma}$ is active under unstable (convective) forcing.

Region 2 – Ocean Interior ($z < -h_{bl}$):

Mixing depends on the local Richardson number for shear-driven instability, on the Brunt-Väisälä frequency for convective instability, and includes a constant background contribution from internal wave breaking.

1.3 Key State Variables

The KPP package maintains the following global state arrays, declared in `KPP.h`:

Variable	Dims	Description	Units
KPPviscAz	(x, y, N_r)	Vertical eddy viscosity	m^2/s
KPPdiffKzT	(x, y, N_r)	Vertical heat diffusivity	m^2/s
KPPdiffKzS	(x, y, N_r)	Vertical salt/tracer diffusivity	m^2/s
KPPghat	(x, y, N_r)	Non-local transport coefficient	s/m^2
KPPhbl	(x, y)	Boundary layer depth	m
KPPfrac	(x, y)	SW fraction heating mixed layer	–
KPPplumefrac	(x, y)	Salt plume fraction penetrating OBL	–

These arrays are computed by `KPP_CALC` and subsequently consumed by the tracer and momentum advection–diffusion solvers each timestep.

2 Governing Equations

This section summarizes the governing equations for vertical mixing in the KPP scheme. The complete mathematical derivation is provided in Large, McWilliams, and Doney (1994).

Variable	Origin	Role / Transformation	Consumer
theta, salt	MITgcm dynamics	Temperature, salinity used in EOS	STATEKPP
DBLOC	STATEKPP	Local buoyancy gradient; smoothed $\rightarrow$ ghat temporarily	bldepth, Ri_iwmix
Ritop	STATEKPP via KPP_CALC	Buoyancy deficit $\times$ depth; numerator of bulk Ri	bldepth
TTALPHA, SSBETA	STATEKPP	Expansion coefficients; used for bo, bosol, KPP_DOUBLEDIFF	KPP_FORCING_SURF
ustar	KPP_FORCING_SURF	Surface friction velocity; enters all turbulent scale calculations	bldepth, blmix
bo	KPP_FORCING_SURF	Total (non-radiative) surface buoyancy flux	bldepth
bosol	KPP_FORCING_SURF	Radiative surface buoyancy flux; added to bsfsc	bldepth
shsq	KPP_CALC (from uVel, vVel)	Local vertical shear; denominator of interior Ri	Ri_iwmix

Variable	Origin	Role / Transformation	Consumer
dVsq	KPP_FORCING_SURFACE	Surface-ref. shear; denominator of bulk Ri	bldepth
diffusKzS/T	CALC_3D_DIFFUSIVITY ± KPP_DOUBLEDIFF	Background + double-diffusion diffusivities; used as interior baseline in Ri_iwmix and as matching value in blmix	Ri_iwmix, blmix
diffus(:, :, 1:3)	Ri_iwmix	Interior mixing coefficients; overwritten in BL above kbl	KPPMIX (combine step)
hbl, kbl	bldepth	OBL depth and index; control BL/interior partitioning	blmix, enhance, KPP_CALC
bfsfc	bldepth	Effective surface buoyancy forcing at $z = -h_{bl}$ including SW	blmix (via wscale)
blmc(:, :, 1:3)	blmix → enhance	BL mixing coefficients; blend with interior at OBL base	KPPMIX (combine step)
ghat	blmix (zeroed below kbl)	Non-local transport coefficient	KPP_TRANSPORT_T/S/PTR
KPPviscAz	KPP_CALC (from vddiff(:, :, :, 1))	Final 3D viscosity; averaged to u,v grid	KPP_CALC_VISC
KPPdiffKzT	KPP_CALC (from vddiff(:, :, :, 3))	Final 3D temperature diffusivity	KPP_CALC_DIFF_T, KPP_TRANSPORT_T
KPPdiffKzS	KPP_CALC (from vddiff(:, :, :, 2))	Final 3D salt/tracer diffusivity	KPP_CALC_DIFF_S/PTR, KPP_TRANSPORT_T
KPPghat	KPP_CALC (from ghat)	Final 3D non-local coefficient	KPP_TRANSPORT_T/S/PTR
KPPhbl	KPP_CALC (from hbl)	OBL depth; used for KPPfrac, KPPplumefrac	KPP_OUTPUT, user diagnostics
KPPfrac	KPP_CALC via SWFRAC	SW fraction penetrating OBL; modifies effective surface Q_T	KPP_TRANSPORT_T
KappaRT, KappaRS	KPP_CALC_DIFF_T	Diffusivity passed to tracer implicit solver	CALC_DIFFUSIVITY
KappaRU, KappaRV	KPP_CALC_VISC	Viscosity passed to momentum implicit solver	CALC_DIFFUSIVITY
df	KPP_TRANSPORT_T/S/PTR	Non-local diffusive flux added to tracer tendency	GAD_DIFF_X family

Table 2: Evolution of key variables through the KPP computation chain.

3 Package Architecture and Call Flow

3.1 Initialization Sequence

The KPP package initialization occurs in two stages, called during `INITIALISE_FIXED` and `INITIALISE_VARIA` respectively.

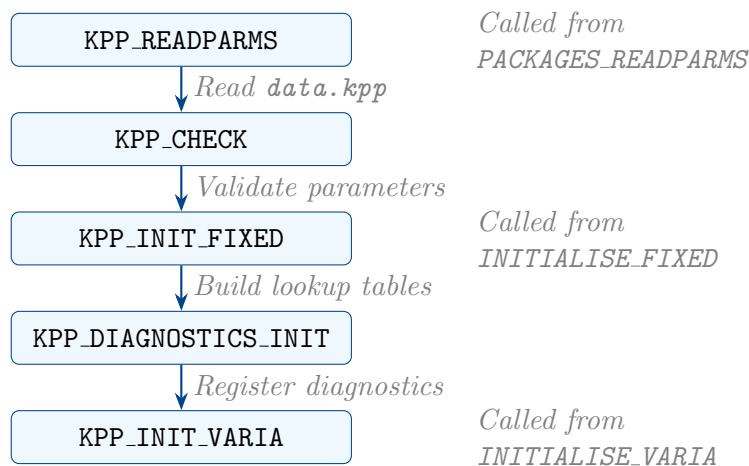


Figure 1: KPP initialization call sequence.

3.2 Per-Timestep Execution Flow

The central driver routine `KPP_CALC` is called once per tile (bi, bj) at a frequency controlled by `kpp_freq`. The downstream interface routines are called at every model timestep.

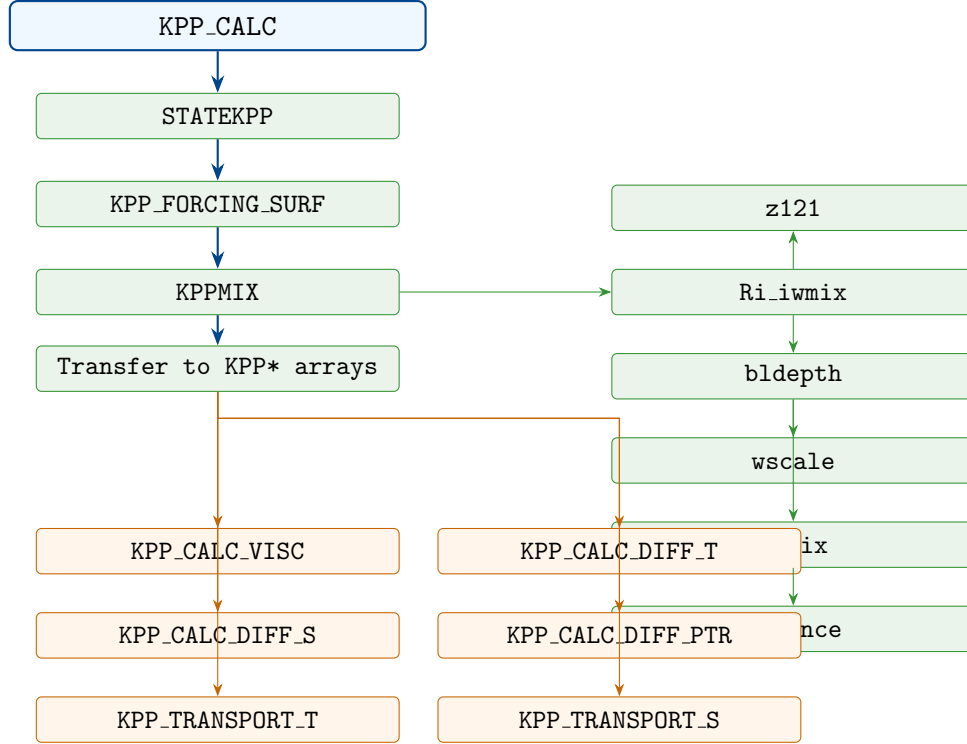


Figure 2: Per-timestep KPP call hierarchy. **Blue**: driver. **Green**: physics subroutines (called within KPP_CALC). **Orange**: interface routines (called every timestep from main model).

3.3 Execution Frequency

A key efficiency feature is that the full KPP calculation is only recomputed when required. In `kpp_calc.F`, the computation is gated by a time-comparison check:

```

1  c      Check to see if new vertical mixing coefficient should
2  c      be computed now?
3  IF ( DIFFERENT_MULTIPLE(kpp_freq,myTime,deltaTClock)
4  1      .OR. myTime .EQ. startTime ) THEN
5      ...
6      CALL KPPMIX ( ... )
7      ...
8  ENDIF

```

Listing 1: `kpp_calc.F` – Conditional recomputation of KPP fields.

Design implication: Setting `kpp_freq > deltaTClock` in `data.kpp` allows KPP to be computed less frequently than the dynamical timestep, e.g. once per hour, to reduce cost. The KPP arrays (`KPPviscAz`, `KPPdiffKzT`, etc.) are held constant between recomputations.

4 Parameter Initialization

4.1 KPP_READPARMS – Reading data.kpp

All tunable KPP parameters are read from the Fortran namelist KPP_PARM01 within `data.kpp`. The complete set of parameters, their defaults, and their physical meaning is given in Table 3.

Parameter	Default	Units	Description
<i>— Control —</i>			
kpp_freq	deltaTClock	s	Recomputation frequency
kpp_dumpFreq	dumpFreq	s	Output dump frequency
KPPwriteState	.FALSE.	–	Write KPP state to file
minKPPhbl	UNSET_RL	m	Minimum allowed h_{bl} (no limit if unset)
<i>— Physical constants —</i>			
epsln	10^{-20}	–	Machine epsilon
phepsi	10^{-10}	–	Physical epsilon (division guard)
epsilon	0.1	–	Nondimensional surface layer fraction
vonk	0.4	–	Von Kármán constant
dB_dz	5.2×10^{-5}	s^{-2}	Max N^2 in mixed layer
<i>— Boundary layer depth (bldepth) —</i>			
Ricr	0.3	–	Critical bulk Richardson number
cekman	0.7	–	Ekman depth coefficient
cmonob	1.0	–	Monin-Obukhov depth coefficient
concv	1.8	–	Interior buoyancy frequency ratio
hbf	1.0	–	Solar radiation depth fraction
<i>— Interior mixing (Ri_iwmix) —</i>			
Riinfty	0.7	–	Shear instability Ri limit
BVSQcon	-2×10^{-5}	s^{-2}	Convection N^2 threshold
difm0	5×10^{-3}	m^2/s	Max shear-instability viscosity
difs0	5×10^{-3}	m^2/s	Max shear-instability scalar diffusivity
dift0	5×10^{-3}	m^2/s	Max shear-instability heat diffusivity
difmcon	0.1	m^2/s	Convective viscosity
difscon	0.1	m^2/s	Convective scalar diffusivity

Parameter	Default	Units	Description
diftcon	0.1	m ² /s	Convective heat diffusivity
— <i>Double diffusion (KPP_DOUBLEDIFF)</i> —			
Rrho0	1.9	—	Density ratio limit
dsfmax	10 ⁻³	m ² /s	Max salt-fingering diffusivity
— <i>Boundary layer mixing (blmix)</i> —			
cstar	10.0	—	Non-local transport proportionality coeff.

Table 3: KPP run-time parameters read from `data.kpp`.

4.2 KPP_INIT_FIXED – Time-Invariant Initialization

This subroutine performs two critical one-time tasks:

4.2.1 Construction of Turbulent Velocity Scale Lookup Tables

Because the turbulent velocity scale functions $w_m(\sigma)$ and $w_s(\sigma)$ are computed very frequently (inside the inner loops of `bldepth` and `blmix`), they are pre-tabulated as functions of the stability parameter $\hat{\zeta} = \kappa\sigma h_{bl} B_f$ and the friction velocity u_* .

The lookup table dimensions are defined with `nni`=890 and `nnj`=480, giving arrays `[0 : nni + 1, 0 : nnj + 1] = [0 : 891, 0 : 481]` with total size `[892, 482]`, spanning:

$$\hat{\zeta} \in [\text{zmin}, \text{zmax}] = [-4 \times 10^{-7}, 0] \quad \text{m}^3/\text{s}^3 \quad (1)$$

$$u_* \in [\text{umin}, \text{umax}] = [0, 4 \times 10^{-2}] \quad \text{m/s} \quad (2)$$

The table entries follow different formulas for stable/unstable regimes:

```

1      DO i = 0, nni + 1
2          zehat = deltaz*i + zmin
3          DO j = 0, nnj + 1
4              usta = deltau*j + umin
5              zeta = zehat / max(phepsi, usta**3)
6              IF (zehat .GE. 0.) THEN
7  c          Stable: both um and ws follow Monin-Obukhov
8              wmt(i,j) = vonk*usta/(1. + conc1*zeta)
9              wst(i,j) = wmt(i,j)
10             ELSE
11                 IF (zeta .GT. zetam) THEN
12  c          Unstable, intermediate regime for momentum
13                 wmt(i,j) = vonk*usta*(1. - conc2*zeta)**p25
14                 ELSE
15  c          Unstable, deep convective regime for momentum
16                 wmt(i,j) = vonk*(conam*usta**3 - concm*zehat)**p33
17             ENDIF

```

```

18         IF (zeta .GT. zetas) THEN
19 c           Unstable, intermediate regime for scalars
20             wst(i,j) = vonk*usta*SQRT(1. _d 0 - conc3*zeta)
21         ELSE
22 c           Unstable, deep convective regime for scalars
23             wst(i,j) = vonk*(conas*usta**3 - concs*zehat)**p33
24         ENDIF
25     ENDIF
26 ENDDO
27 ENDDO

```

Listing 2: `kpp_init_fixed.F`, lines 135–156 – Construction of `wmt/wst` lookup tables.

The mathematical forms are, for the *unstable* case ($\hat{\zeta} \leq 0$):

$$w_m(\sigma) = \begin{cases} \kappa u_* (1 - c_2 \zeta)^{1/4} & \zeta_m < \zeta \leq 0 \\ \kappa (c_{am} u_*^3 - c_m \hat{\zeta})^{1/3} & \zeta \leq \zeta_m \end{cases} \quad (3)$$

$$w_s(\sigma) = \begin{cases} \kappa u_* \sqrt{1 - c_3 \zeta} & \zeta_s < \zeta \leq 0 \\ \kappa (c_{as} u_*^3 - c_s \hat{\zeta})^{1/3} & \zeta \leq \zeta_s \end{cases} \quad (4)$$

where $\zeta = \hat{\zeta}/u_*^3$ is the stability parameter.

For the *stable* case ($\hat{\zeta} > 0$):

$$w_m = w_s = \frac{\kappa u_*}{1 + c_1 \zeta} \quad (5)$$

4.2.2 Vertical Grid Setup

The vertical grid arrays used internally by KPP (`zgrid`, `hwide`) are mapped from the MITgcm model grid:

```

1     DO k = 1, Nr
2         zgrid(k) = rC(k)      ! Cell center depth (<=0)
3         hwide(k) = drF(k)    ! Layer thickness (>0)
4     ENDDO
5     zgrid(Nrp1) = zgrid(Nr) * 100. ! fictitious bottom layer
6     hwide(Nrp1) = phepsi        ! near-zero width

```

Listing 3: `kpp_init_fixed.F`, lines 115–128 – Mapping of KPP internal grid from MITgcm grid.

Note: The fictitious layer at index `Nrp1` is required for the `blmix` routine, which needs to evaluate diffusivities one level below the deepest wet cell. It is assigned a near-zero width (`phepsi`) to avoid unphysical behavior.

4.3 KPP_INIT_VARIA – Variable Initialization

Initializes all time-varying KPP arrays for each tile. Also computes `nzmax(i,j,bi,bj)`, the deepest wet level index (bottom depth in model levels), by mapping from `kLowC`:

```

1      DO j = 1-OLy, sNy+OLy
2          DO i = 1-OLx, sNx+OLx
3              nzmax(i,j,bi,bj) = kLowC(I,J,bi,bj)
4          ENDDO
5      ENDDO
6      DO k = 1, Nr
7          DO j = 1-OLy, sNy+OLy
8              DO i = 1-OLx, sNx+OLx
9                  KPPghat (i,j,k,bi,bj) = 0.
10                 KPPviscAz (i,j,k,bi,bj) = viscArNr(1)
11                 KPPdiffKzS(i,j,k,bi,bj) = 0.
12                 KPPdiffKzT(i,j,k,bi,bj) = 0.
13             ENDDO
14         ENDDO
15     ENDDO

```

Listing 4: kpp_init_varia.F, lines 43–68 – Array initialization and nzmax computation.

Note that KPPviscAz is initialized to viscArNr(1), the background vertical viscosity, rather than zero. This ensures a physically reasonable value is present before the first KPP computation.

5 Main Driver: KPP_CALC

KPP_CALC (kpp_calc.F) is the top-level interface between the MITgcm ocean model and the KPP mixing routines. It:

1. Checks whether a KPP recomputation is due
2. Calls STATEKPP to compute buoyancy-related diagnostics
3. Optionally smooths density-derived quantities horizontally
4. Constructs surface forcing arrays via KPP_FORCING_SURF
5. Computes local velocity shear squared (shsq)
6. Pre-computes background diffusivities via CALC_3D_DIFFUSIVITY
7. Optionally adds double diffusion via KPP_DOUBLEDIFF
8. Calls KPPMIX to compute boundary layer mixing
9. Transfers results to global arrays with land masking
10. Computes shortwave and salt-plume fraction fields

5.1 Velocity Shear Computation

Two shear-squared quantities are computed within KPP_CALC:

Local shear shsq (for Richardson number in the interior):

$$\text{shsq}_k = \frac{1}{2} [(u_k - u_{k+1})^2 + (u_{k+1,\text{east}} - u_{k+1+1,\text{east}})^2 + (v_k - v_{k+1})^2 + (v_{k,\text{north}} - v_{k+1,\text{north}})^2] \quad (6)$$

This averages the velocity shear from the u,v-grid onto the t-grid.

When KPP_SMOOTH_SHSQ is defined (default), a 121 horizontal filter is applied using contributions from neighboring columns:

```

1 #ifdef KPP_SMOOTH_SHSQ
2     shsq(i,j,k) = p5 * shsq(i,j,k) + p125 * (
3         &      (uVel(i,  jm1,k,bi,bj)-uVel(i,  jm1,kp1,bi,bj)) *
4         &      (uVel(i,  jm1,k,bi,bj)-uVel(i,  jm1,kp1,bi,bj)) +
5         &      ... [8 more neighbor contributions] ...
6         &      (vVel(ip1,jp1,k,bi,bj)-vVel(ip1,jp1,kp1,bi,bj)) *
7         &      (vVel(ip1,jp1,k,bi,bj)-vVel(ip1,jp1,kp1,bi,bj)) )
8 #endif

```

Listing 5: kpp_calc.F, lines ~185–210 – Horizontal smoothing of **shsq** with 121 filter.

The smoothing weights are: center = 0.5, face neighbors = 0.125 each (total = $4 \times 0.125 = 0.5$), summing to 1.

Surface-referenced shear dVs_q (for bulk Richardson number in boundary layer depth search):

$$\text{dVs}_q_k = \frac{1}{2} [(U_{\text{ref}} - u_k)^2 + \dots + (V_{\text{ref}} - v_{k,\text{north}})^2] \quad (7)$$

where U_{ref} , V_{ref} are the surface layer velocities (or an estimated reference velocity if KPP_ESTIMATE_UREF is defined).

5.2 Background Diffusivity and Double Diffusion

Before calling KPPMIX, background diffusivities are computed by calling the standard MIT-gcm routine CALC_3D_DIFFUSIVITY:

```

1     CALL CALC_3D_DIFFUSIVITY(
2     I      bi,bj,1-OLx,sNx+OLx,1-OLy,sNy+OLy,
3     I      GAD_SALINITY, .FALSE., .FALSE.,
4     O      KPPdiffKzS(1-OLx,1-OLy,1,bi,bj),
5     I      myThid)
6     CALL CALC_3D_DIFFUSIVITY(
7     I      bi,bj,1-OLx,sNx+OLx,1-OLy,sNy+OLy,
8     I      GAD_TEMPERATURE, .FALSE., .FALSE.,
9     O      KPPdiffKzT(1-OLx,1-OLy,1,bi,bj),
10    I      myThid)
11 #ifndef EXCLUDE_KPP_DOUBLEDIFF
12     IF ( KPPuseDoubleDiff ) THEN
13         CALL KPP_DOUBLEDIFF(
14         I      TTALPHA, SSBETA,
15         U      KPPdiffKzT(1-OLx,1-OLy,1,bi,bj),
16         U      KPPdiffKzS(1-OLx,1-OLy,1,bi,bj),

```

```

17      I      ikey,1-OLx,sNx+OLx,1-OLy,sNy+OLy,bi,bj,myThid)
18      ENDIF
19 #endif

```

Listing 6: `kpp_calc.F` – Background diffusivity computation and optional double-diffusion enhancement.

These background diffusivities serve two roles: (1) they provide the interior diffusivity baseline upon which shear instability and convective mixing add incremental contributions in `Ri_iwmix`, and (2) they allow the KPP boundary layer diffusivity to match the interior value at $z = -h_{bl}$ (the matching/smoothing condition in `blmix`).

5.3 Result Transfer and Masking

After `KPPMIX` returns, the 1-D internal arrays `vddiff` (indexed as `(x,y,0:Nrp1,mdiff)`) are transferred to the 3-D global arrays with land masking applied:

```

1      DO k = 1, Nr
2          km1 = max(1,k-1)
3          DO j = jMin, jMax
4              DO i = iMin, iMax
5                  KPPviscAz(i,j,k,bi,bj) = vddiff(i,j,k-1,1)
6              &      * maskC(i,j,k,bi,bj) * maskC(i,j,km1,bi,bj)
7                  KPPdiffKzS(i,j,k,bi,bj) = vddiff(i,j,k-1,2)
8              &      * maskC(i,j,k,bi,bj) * maskC(i,j,km1,bi,bj)
9                  KPPdiffKzT(i,j,k,bi,bj) = vddiff(i,j,k-1,3)
10             &      * maskC(i,j,k,bi,bj) * maskC(i,j,km1,bi,bj)
11             KPPghat(i,j,k,bi,bj) = ghat(i,j,k)
12             &      * maskC(i,j,k,bi,bj) * maskC(i,j,km1,bi,bj)
13         ENDDO
14     ENDDO
15 ENDDO

```

Listing 7: `kpp_calc.F` – Transfer of `KPPMIX` output to global KPP arrays with masking.

The double masking (`maskC(k) × maskC(km1)`) ensures that diffusivities are zero at interfaces adjacent to land or seafloor cells.

6 Stratification and Buoyancy: STATEKPP

Located in `kpp_routines.F`, `STATEKPP` computes all density-related input fields needed by the KPP scheme. It calls the standard MITgcm equation-of-state routines `FIND_RHO_2D`, `FIND_ALPHA`, and `FIND_BETA`.

6.1 Outputs

Variable	Definition	Units
RH01	Surface layer potential density: $\rho_1 = \rho(T_1, S_1, p_1) + \rho_0$	kg/m ³
TTALPHA	Thermal expansion: $\alpha = \partial\rho/\partial T$	kg/m ³ /°C
SSBETA	Haline contraction: $\beta = \partial\rho/\partial S$	kg/m ³ /(g/kg)
DBLOC(<i>k</i>)	Local buoyancy gradient across interface <i>k</i> : $\delta b_{\text{loc}}(k) = g(\rho_k - \rho_{k-1})/\rho_k$	m/s ²
DBSFC(<i>k</i>)	Buoyancy difference w.r.t. surface: $\delta b_{\text{sfc}}(k) = g(\rho_k - \rho_{1k})/\rho_k$	m/s ²

6.2 Key Density Calls

For each level $k \geq 2$, three density evaluations are made:

```

1  c      RHOK   = rho( T(k),   S(k),   pressure(k) )
2          CALL FIND_RHO_2D( ..., k, theta(:, :, k, :, :),
3      0      RHOK, k, bi, bj, myThid )
4
5  c      RHOKM1 = rho( T(k-1), S(k-1), pressure(k) ) -- at level k!
6  c      This evaluates the density of the overlying layer at the
7  c      pressure of the current layer (adiabatic displacement)
8          CALL FIND_RHO_2D( ..., k, theta(:, :, k-1, :, :),
9      0      RHOKM1, k-1, bi, bj, myThid )
10
11 c      RH01K  = rho( T(1),   S(1),   pressure(k) ) -- surface parcel
12 c      displaced adiabatically to level k (for bulk Richardson number)
13          CALL FIND_RHO_2D( ..., k, theta(:, :, 1, :, :),
14      0      RH01K, 1, bi, bj, myThid )

```

Listing 8: `kpp_routines.F` (STATEKPP) – Density evaluations for buoyancy gradient computation.

This gives the physically correct definition of static stability (accounting for adiabatic compressibility) and bulk density difference:

$$\delta b_{\text{loc}}(k-1) = g \frac{\rho_k - \rho_{k-1}}{\rho_k + \rho_0} \quad (\text{local buoyancy gradient}) \quad (8)$$

$$\delta b_{\text{sfc}}(k) = g \frac{\rho_k - \rho_{1k}}{\rho_k + \rho_0} \quad (\text{surface-referenced buoyancy deficit}) \quad (9)$$

Note that DBLOC(*k*) is defined at the *bottom* of cell *k* (i.e., at the interface between cells *k* and *k*+1 in MITgcm indexing), not at the top. This has important implications for masking: the correct mask is `maskC(k) × maskC(k+1)`.

7 Surface Forcing: KPP_FORCING_SURF

Located in `kpp_forcing_surf.F`, this subroutine converts model surface forcing fields (in MITgcm units) into the three surface forcing quantities required by KPP.

7.1 Friction Velocity

The surface friction velocity u_* is derived from the wind stress:

$$u_* = \sqrt{\sqrt{\tau_x^2 + \tau_y^2} / \rho_{\text{surf}}} \quad (10)$$

where τ_x, τ_y are the zonal and meridional wind stresses (`surfaceForcingU`, `surfaceForcingV` in MITgcm). The implementation averages the u,v-grid stresses onto the t-grid:

```

1      DO j = jmin, jmax
2          jp1 = j + 1
3          DO i = imin, imax
4              ip1 = i+1
5              work3(i,j) =
6              &      (surfForcU(i,j,bi,bj) + surfForcU(ip1,j,bi,bj)) *
7              &      (surfForcU(i,j,bi,bj) + surfForcU(ip1,j,bi,bj)) +
8              &      (surfForcV(i,j,bi,bj) + surfForcV(i,jp1,bi,bj)) *
9              &      (surfForcV(i,j,bi,bj) + surfForcV(i,jp1,bi,bj))
10             ENDDO
11         ENDDO
12         epsLocSq = phepsi*phepsi*drF(1)*drF(1)
13         epsLoc = SQRT ( p5*phepsi*drF(1) )
14         DO j = jmin, jmax
15             DO i = imin, imax
16                 IF ( work3(i,j) .lt. epsLocSq ) THEN
17                     ustar(i,j) = epsLoc
18                 ELSE
19                     tempVar2 = SQRT( work3(i,j) ) * p5
20                     ustar(i,j) = SQRT( tempVar2 )
21                 ENDIF
22             ENDDO
23         ENDDO

```

Listing 9: `kpp_forcing_surf.F`, lines ~130–155 – Computation of friction velocity `ustar`.

7.2 Surface Buoyancy Fluxes

Turbulent buoyancy flux `bo`:

$$B_0 = -\frac{g}{\rho_{\text{surf}}} [\alpha (Q_T + Q_{\text{ice}}) + \beta (E - P - R) \cdot S_{\text{surf}}] \quad (11)$$

```

1      DO j = jmin, jmax
2          DO i = imin, imax
3              bo(i,j) = - gravity *
4              &      ( TTALPHA(i,j,1) * (surfForcT(i,j,bi,bj) +

```

```

5      &      surfFor cTice(i,j,bi,bj)) +
6      &      SSBETA(i,j,1) * surfFor cS(i,j,bi,bj) )
7      &      / rhoSurf(i,j)
8      ENDDO
9      ENDDO

```

Listing 10: kpp_forcing_surf.F – Turbulent surface buoyancy forcing.

Radiative (shortwave) buoyancy flux bosol:

$$B_{0,\text{sol}} = \frac{g \alpha Q_{\text{sw}}}{C_p \rho_0 \rho_{\text{surf}}} \quad (12)$$

This is the component of surface buoyancy forcing due to shortwave radiation. It is separated from `bo` because it penetrates below the surface and its contribution to the mixed layer heat budget depends on the depth profile of absorption.

Haline buoyancy flux from salt plumes boplume: When `ALLOW_SALT_PLUME` is defined:

$$B_{\text{plume}} = -\frac{g \beta F_{\text{SP}}}{\rho_0^2} \quad (13)$$

where F_{SP} is the downward salt flux from brine rejection.

7.3 Surface-Referenced Velocity Shear dVsq

Without `KPP_ESTIMATE_UREF`, the shear is referenced to the top model level:

$$\text{dVsq}(k) = \frac{1}{2} \sum_{\text{neighbors}} [(U_1 - U_k)^2 + (V_1 - V_k)^2] \quad (14)$$

8 Interior Mixing: Ri_iwmix

`Ri_iwmix` computes vertical diffusivities at all interfaces $k = 1, \dots, N_r$ due to three mechanisms: (1) background internal wave activity, (2) shear instability, and (3) static instability (convection).

8.1 Richardson Number Computation

The local gradient Richardson number is computed from the smoothed buoyancy gradient and the local velocity shear:

$$\text{Ri}_{\text{loc}}(k) = \frac{\delta b_{\text{loc,sm}}(k) \cdot (z_k - z_{k+1})}{\max(\text{shsq}(k), \varepsilon)} \quad (15)$$

where `dblocSm` is the horizontally smoothed version of `dbloc` (stored in `ghat` temporarily to save memory):

```

1      DO ki = 1, Nr
2          DO i = 1, imt
3              IF      (kmtj(i) .LE. 1          ) THEN
4                  diffus(i,ki,1) = 0.
5                  diffus(i,ki,2) = 0.
6              ELSEIF (ki      .GE. kmtj(i)) THEN
7  c          At/below seafloor: use last valid value
8                  diffus(i,ki,1) = diffus(i,ki-1,1)
9                  diffus(i,ki,2) = diffus(i,ki-1,2)
10             ELSE
11  c          Interior: compute Ri from smoothed dbloc / shsq
12                  diffus(i,ki,1) = dblocSm(i,ki) * (zgrid(ki)-zgrid(ki+1))
13                  &      / MAX( Shsq(i,ki), phepsi )
14                  diffus(i,ki,2) = dbloc(i,ki) / (zgrid(ki)-zgrid(ki+1))
15             ENDIF
16         ENDDO
17     ENDDO

```

Listing 11: kpp_routines.F (Ri_iwmx) – Local Richardson number computation.

Here `diffus(:, :, 1)` temporarily stores the Richardson number, and `diffus(:, :, 2)` stores N^2 (Brunt-Väisälä frequency squared). These are overwritten with the actual diffusivities in the next step.

8.2 Mixing Functions

Two nonlinear functions of the Richardson number control the strength of mixing:

Convective mixing function f_{con} :

$$f_{\text{con}} = \left[1 - \left(\min \left(\frac{N_{\text{con}}^2 - N^2}{N_{\text{con}}^2}, 1 \right) \right)^2 \right]^3 \quad (16)$$

where $N_{\text{con}}^2 = \text{BVSQcon} = -2 \times 10^{-5} \text{ s}^{-2}$. This is active ($f_{\text{con}} > 0$) only when $N^2 < 0$, i.e. the water column is statically unstable.

Shear instability function f_{Ri} :

$$f_{\text{Ri}} = \left[1 - \left(\min \left(\frac{\text{Ri}}{\text{Ri}_{\infty}}, 1 \right) \right)^2 \right]^3 \quad (17)$$

where $\text{Ri}_{\infty} = \text{Riinfy} = 0.7$. This goes from 1 (at $\text{Ri} = 0$) to 0 (at $\text{Ri} \geq \text{Ri}_{\infty}$).

```

1      DO ki = 1, Nr
2          DO i = 1, imt
3  c          f of Brunt-Vaisala squared for convection
4              Rig = MAX ( diffus(i,ki,2) , BVSQcon )
5              ratio = MIN ( (BVSQcon - Rig) / BVSQcon, c1 )
6              fcon = c1 - ratio * ratio

```

```

7      fcon = fcon * fcon * fcon
8
9  c      f of smooth Ri for shear instability
10     Rig = MAX ( diffus(i,ki,1), c0 )
11     ratio = MIN ( Rig / Riinfy , c1 )
12     fRi = c1 - ratio * ratio
13     fRi = fRi * fRi * fRi
14
15  c      Final diffusivities
16     kp1 = MIN(ki+1,Nr)
17     diffus(i,ki,1) = viscArNr(1) + fcon*difmcon + fRi*difm0
18     diffus(i,ki,2) = diffusKzS(i,kp1)+fcon*difscon+fRi*difs0
19     diffus(i,ki,3) = diffusKzT(i,kp1)+fcon*diftcon+fRi*dift0
20     ENDDO
21     ENDDO

```

Listing 12: kpp_routines.F (Ri_iwmix) – Evaluation of mixing functions and diffusivities.

The resulting interior diffusivities at each interface are:

$$\kappa_m(k) = \nu_{bg} + f_{con} \cdot \nu_{con} + f_{Ri} \cdot \nu_{shear} \quad (18)$$

$$\kappa_S(k) = \kappa_{S,bg}(k+1) + f_{con} \cdot \kappa_{S,con} + f_{Ri} \cdot \kappa_{S,shear} \quad (19)$$

$$\kappa_T(k) = \kappa_{T,bg}(k+1) + f_{con} \cdot \kappa_{T,con} + f_{Ri} \cdot \kappa_{T,shear} \quad (20)$$

Background diffusivity indexing: The background diffusivities (`diffusKzS(i,kp1)`, `diffusKzT(i,kp1)`) are taken at level $k+1$ (one level below the current interface). This means the background at the bottom interface of cell k uses the background diffusivity representative of the cell below.

8.3 Vertical Smoothing of Ri (z121)

When `ALLOW_KPP_VERTICALLY_SMOOTH` is defined, the Richardson number stored in `diffus(:, :, 1)` is smoothed vertically using the 121 filter `z121`. This filter applies the stencil $[1/4, 1/2, 1/4]$ in the vertical, but *only to grid points where* $0 \leq Ri \leq Ri_\infty$ (within the turbulent mixing range). This prevents smoothing from contaminating convectively unstable regions or quiescent stratified regions.

9 Boundary / Surface Layer Mixing

The oceanic boundary layer (OBL) mixing is computed through a sequence of four subroutines that diagnose the boundary layer depth, compute turbulent velocity scales, construct shape functions, and enhance the profiles at the boundary layer interface.

9.1 Boundary Layer Depth Diagnosis: bldepth

bldepth determines the depth of the oceanic planetary boundary layer h_{bl} by searching downward for the level at which the bulk Richardson number first exceeds the critical value $Ri_{cr} = 0.3$.

9.2 Bulk Richardson Number

The bulk Richardson number at level k is:

$$Ri_b(k) = \frac{Ritop(k)}{dVsq(k) + V_t^2(k)} = \frac{(z_{ref} - z_k) \delta b_{sfc}(k)}{(U_{ref} - U_k)^2 + (V_{ref} - V_k)^2 + V_t^2(k)} \quad (21)$$

where the turbulent shear term V_t^2 accounts for the unresolved contribution of sub-grid turbulence to the denominator:

$$V_t^2(k) = -z_k \cdot w_s \cdot \sqrt{|N_k^2|} \cdot V_{tc} \quad (22)$$

```

1      DO i = 1, imt
2      c      Estimate N^2 at this level from adjacent dbloc values
3      bvsq = p5 *
4      1      ( dbloc(i,kl-1) / (zgrid(kl-1)-zgrid(kl  )) +
5      2      dbloc(i,kl  ) / (zgrid(kl  )-zgrid(kl+1)))
6      IF (bvsq .EQ. 0. _d 0) THEN
7      vtsq = 0. _d 0
8      ELSE
9      c      Turbulent shear contribution
10     vtsq = -zgrid(kl) * ws(i) * SQRT(ABS(bvsq)) * Vtc
11     ENDIF
12     c      Bulk Richardson number
13     tempVar1 = dvsq(i,kl) + vtsq
14     tempVar2 = MAX(tempVar1, phepsi)
15     Rib(i,kl) = Ritop(i,kl) / tempVar2
16     ENDDO

```

Listing 13: kpp.routines.F (bldepth) – Turbulent shear term and bulk Richardson number.

9.3 Linear Interpolation for hbl

Once the level $k = k_{bl}$ where Ri_b first exceeds Ri_{cr} is identified, h_{bl} is obtained by linear interpolation:

$$h_{bl} = -z_{k-1} + (z_{k-1} - z_k) \frac{Ri_{cr} - Ri_b(k-1)}{Ri_b(k) - Ri_b(k-1)} \quad (23)$$

```

1      DO i = 1, imt
2      kl = kbl(i)
3      c      linearly interpolate to find hbl where Rib = Ricr
4      IF (kl.GT.1 .AND. kl.LT.kmtj(i)) THEN
5      tempVar1 = (Rib(i,kl)-Rib(i,kl-1))

```

```

6          hbl(i) = -zgrid(kl-1) + (zgrid(kl-1)-zgrid(kl)) *
7      1          (Ricr - Rib(i,kl-1)) / tempVar1
8          ENDIF
9      ENDDO

```

Listing 14: kpp_routines.F (bldepth) – Linear interpolation to find exact hbl.

9.4 Stability Limits on hbl

Under stable surface forcing ($B_f > 0$), hbl is additionally limited by Ekman depth and Monin-Obukhov depth:

$$h_{\text{Ekman}} = c_{\text{Ekman}} \frac{u_*}{|f|} \quad (24)$$

$$h_{\text{MO}} = c_{\text{MO}} \frac{u_*^3}{\kappa B_f} \quad (25)$$

$$h_{\text{bl}} = \min(h_{\text{bl}}, \min(h_{\text{Ekman}}, h_{\text{MO}})) \quad (\text{stable conditions only}) \quad (26)$$

```

1      IF ( LimitHblStable ) THEN
2          DO i = 1, imt
3              IF (bfsfc(i) .GT. 0.0) THEN
4                  hekman = cekman * ustar(i) / MAX(ABS(Coriol(i)), phepsi)
5                  hmonob = cmonob * ustar(i)*ustar(i)*ustar(i)
6              & / vonk / bfsfc(i)
7                  hlimit = stable(i) * MIN(hekman, hmonob)
8              & + (stable(i)-1.) * zgrid(Nr)
9                  hbl(i) = MIN(hbl(i), hlimit)
10             ENDIF
11         ENDDO
12     ENDIF

```

Listing 15: kpp_routines.F (bldepth) – Ekman and Monin-Obukhov depth limits.

Finally, hbl is constrained from below by minKPPhbl (default: depth of first model level center):

$$h_{\text{bl}} = \max(h_{\text{bl}}, h_{\text{bl,min}}) \quad (27)$$

9.5 Case A / Case B Classification

After determining hbl, the subroutine classifies each column as Case A or Case B, which determines how the matching between boundary layer and interior diffusivities is handled in blmix:

$$\text{caseA}(i) = \begin{cases} 1 & \text{if } -z_{k_{\text{bl}}} - \frac{1}{2}h_{k_{\text{bl}}} > h_{\text{bl}} \quad (\text{hbl is within cell kbl}) \\ 0 & \text{otherwise (hbl is at cell boundary)} \end{cases} \quad (28)$$

9.6 Turbulent Velocity Scales: wscale

Given σ (normalized depth), h_{bl} , u_* , and the total surface buoyancy flux B_f , **wscale** returns the turbulent velocity scales w_m and w_s by bilinear interpolation in the pre-computed lookup tables.

9.7 Bilinear Interpolation

```

1      DO i = 1, imt
2          zehat = vonk*sigma(i)*hbl(i)*bfsfc(i)
3
4          IF (zehat .LE. zmax) THEN
5      c      Use lookup table
6          zdifff = zehat - zmin
7          iz     = MIN( MAX(INT( zdifff / deltaz ), 0), nni )
8          izp1   = iz + 1
9          udifff = ustar(i) - umin
10         ju     = MIN( MAX(INT( udifff / deltau ), 0), nnj )
11         jup1   = ju + 1
12
13         zfrac  = zdifff / deltaz - float(iz)
14         ufrac  = udifff / deltau - float(ju)
15         fzfrac = 1. - zfrac
16
17     c      Bilinear interpolation for wm
18         wam    = fzfrac * wmt(iz, jup1) + zfrac * wmt(izp1, jup1)
19         wbm    = fzfrac * wmt(iz, ju ) + zfrac * wmt(izp1, ju )
20         wm(i) = (1.-ufrac) * wbm      + ufrac * wam
21     ELSE
22     c      Stable conditions outside table range
23         u3 = ustar(i) * ustar(i) * ustar(i)
24         wm(i) = vonk * ustar(i) * u3 / (u3 + conc1 * zehat)
25         ws(i) = wm(i)
26     ENDIF
27 ENDDO

```

Listing 16: kpp_routines.F (wscale) – Bilinear interpolation in wm/ws lookup tables.

9.8 Boundary Layer Mixing Coefficients: blmix

blmix computes turbulent diffusivities and viscosities *within* the boundary layer using a cubic polynomial profile.

9.9 Shape Function $G(\sigma)$

The KPP boundary layer mixing profile has the form:

$$K(\sigma) = h_{bl} w(\sigma) \sigma G(\sigma) \quad (29)$$

where the shape function $G(\sigma)$ is a cubic polynomial in σ :

$$G(\sigma) = a_1 + a_2 g_1 + a_3 g'_1 \quad (30)$$

with $a_1 = \sigma - 2$, $a_2 = 3 - 2\sigma$, $a_3 = \sigma - 1$, and the matching conditions at $\sigma = 1$ (i.e. at $z = -h_{bl}$):

$$g_1 = K(h_{bl})/(h_{bl} w(1)), \quad g'_1 = \frac{dK/d\sigma|_{\sigma=1}}{w(1)} + f_1 K(h_{bl}) \quad (31)$$

The constant f_1 stabilizes the profile under stable forcing:

$$f_1 = c_1 \frac{B_f}{\max(u_*^4, \varepsilon)} \quad (32)$$

```

1      DO i = 1, imt
2          sig = (-zgrid(ki) + 0.5 * hwide(ki)) / hbl(i)
3          a1 = sig - 2.
4          a2 = 3. - 2. * sig
5          a3 = sig - 1.
6
7      c      Dimensionless shape functions
8          Gm = a1 + a2 * gat1m(i) + a3 * dat1m(i)
9          Gs = a1 + a2 * gat1s(i) + a3 * dat1s(i)
10         Gt = a1 + a2 * gat1t(i) + a3 * dat1t(i)
11
12     c      Boundary layer diffusivities
13         blmc(i,ki,1) = hbl(i) * wm(i) * sig * (1. + sig * Gm)
14         blmc(i,ki,2) = hbl(i) * ws(i) * sig * (1. + sig * Gs)
15         blmc(i,ki,3) = hbl(i) * ws(i) * sig * (1. + sig * Gt)
16     ENDDO

```

Listing 17: kpp_routines.F (blmix) – Shape function and boundary layer diffusivity computation.

The matching conditions g_1 (gat1m/s/t) and g'_1 (dat1m/s/t) are computed at the top of blmix by evaluating the interior diffusivities and their gradients at $z = -h_{bl}$:

```

1      c      Find diffusivities and derivative at hbl
2          delhat = 0.5*hwide(kn) - zgrid(kn) - hbl(i)
3          R      = 1.0 - delhat / hwide(kn)
4      c      Gradient of interior viscosity at kn interface
5          dvdzup = (diffus(i,kn-1,1) - diffus(i,kn ,1)) / hwide(kn)
6          dvdzdn = (diffus(i,kn ,1) - diffus(i,kn+1,1)) / hwide(kn+1)
7          viscp  = 0.5 * ( (1.-R) * (dvdzup + ABS(dvdzup)) +
8              1      R * (dvdzdn + ABS(dvdzdn)) )
9          visch  = diffus(i,kn,1) + viscp * delhat
10
11     c      Matching conditions
12         gat1m(i) = visch / hbl(i) / wm(i)
13         dat1m(i) = -viscp / wm(i) + f1 * visch

```

Listing 18: kpp_routines.F (blmix) – Computation of matching conditions at $z = -h_{bl}$.

9.10 Non-local Transport Term

Under unstable (convective) surface forcing, a counter-gradient non-local transport term $\hat{\gamma}$ is activated:

$$\hat{\gamma}(k) = (1 - s_{\text{stable}}) \frac{c_g}{\max(w_s(\sigma_k) h_{\text{bl}}, \varepsilon)} \quad (33)$$

where $c_g = c^* \kappa (c_s \kappa \epsilon)^{1/3}$ (computed in KPP_INIT_FIXED) and $s_{\text{stable}} \in \{0, 1\}$ indicates stable/unstable forcing.

```

1  c          Non-local transport term = ghat * <ws>o
2          tempVar = ws(i) * hbl(i)
3          ghat(i,ki) = (1.-stable(i)) * cg
4  &          / MAX(phepsi,tempVar)

```

Listing 19: kpp_routines.F (blmix) – Non-local transport coefficient.

The non-local flux adds a term to the vertical diffusive flux of any scalar ϕ :

$$F_\phi(k) = -\kappa_S(k) \left[\frac{\partial \phi}{\partial z} - \hat{\gamma}(k-1) \langle w' \phi' \rangle_s \right] \quad (34)$$

where $\langle w' \phi' \rangle_s$ is the surface flux of ϕ . This term is applied in KPP_TRANSPORT_T, _S, and PTR.

9.11 Interface Enhancement: enhance

At the interface $k = k_{\text{bl}} - 1$ (the cell just above the OBL base), **enhance** modifies the boundary layer diffusivity to ensure smooth continuity with the interior diffusivity. Without this correction there would be a kink in the diffusivity profile at h_{bl} .

The enhancement blends the boundary layer value computed at the $k_{\text{bl}} - 1$ level (**dkm1**) with the diffusivity at the $k_{\text{bl}} - 1/2$ interface:

$$\kappa^*(k_{\text{bl}} - 1) = (1 - \delta) \kappa_{\text{int}}(k_{\text{bl}} - 1) + \delta [(1 - \delta)^2 \kappa_{\text{BL}}(k_{\text{bl}} - 1) + \delta^2 \kappa_{k_{\text{bl}} - 1/2}] \quad (35)$$

where $\delta = (h_{\text{bl}} + z_{k_{\text{bl}}-1}) / (z_{k_{\text{bl}}-1} - z_{k_{\text{bl}}})$ is the fractional distance of **hbl** within the cell:

```

1  DO i = 1, imt
2      ki = kbl(i)-1
3      IF ((ki .ge. 1) .AND. (ki .LT. Nr)) THEN
4          delta = (hbl(i) + zgrid(ki)) / (zgrid(ki) - zgrid(ki+1))
5          DO md = 1, mdiff
6              dkmp5 = casea(i) * diffus(i,ki,md) +
7  1          (1.- casea(i)) * blmc(i,ki,md)
8              dstar = (1.- delta)**2 * dkm1(i,md)
9  &          + delta**2 * dkmp5
10             blmc(i,ki,md) = (1.- delta)*diffus(i,ki,md)
11  &          + delta*dstar
12         ENDDO
13         ghat(i,ki) = (1.- casea(i)) * ghat(i,ki)
14     ENDF

```

15 ENDDO

Listing 20: `kpp_routines.F` (`enhance`) – Diffusivity enhancement at the OBL base.

10 Core Diffusivity and Prognostic Update: KPPMIX

KPPMIX is the master driver for the one-dimensional KPP column calculations. It calls the four core physics routines in sequence and combines their outputs.

The data flow through KPPMIX is:

1. `Ri_iwmix` → interior mixing coefficients in `diffus(:, :, 1:3)`
2. Set seafloor and sub-bottom values to zero in `diffus`
3. `bldepth` → boundary layer depth `hbl`, stability flags, bulk Richardson numbers
4. `blmix` → boundary layer mixing `blmc` and non-local term `ghat`
5. `enhance` → smooth diffusivity at $z = -h_{bl}$
6. Combine: use `blmc` above `kbl`, `diffus` below; zero `ghat` below `kbl`

```

1      DO k = 1, Nr
2          DO i = 1, imt
3              IF (k .LT. kbl(i)) THEN
4 c          Within boundary layer: use blmix result
5              diffus(i,k,1) = MAX ( blmc(i,k,1), viscArNr(1) )
6              diffus(i,k,2) = MAX ( blmc(i,k,2), diffusKzS(i,Nr) )
7              diffus(i,k,3) = MAX ( blmc(i,k,3), diffusKzT(i,Nr) )
8          ELSE
9 c          Below boundary layer: ghat is zero
10             ghat(i,k) = 0. _d 0
11         ENDIF
12     ENDDO
13 ENDDO

```

Listing 21: `kpp_routines.F` (KPPMIX) – Final combination of interior and boundary layer diffusivities.

Note the use of `MAX`: this ensures the boundary layer diffusivity is never less than the background viscosity (`viscArNr(1)`) or the deep interior diffusivity. This is a numerical safeguard for shallow mixed layers.

11 Additional Mixing Processes

Beyond the primary shear-driven and convective mixing mechanisms, KPP includes parameterizations for double diffusion, shortwave penetration, and salt plume effects.

11.1 Double Diffusion: KPP_DOUBLEDIFF

When `KPPuseDoubleDiff = .TRUE.`, double-diffusive contributions are added to the background diffusivities *before* `KPPMIX` is called. Two regimes are handled:

11.2 Salt Fingering ($\alpha dT > \beta dS > 0$)

This occurs when warm salty water overlies cool fresh water (typical of subtropical thermoclines). The density ratio is:

$$R_\rho = \frac{\alpha \Delta T}{\beta \Delta S} \in (1, R_{\rho 0}) \quad (36)$$

The additional diffusivities (LMD94, Eqs. 31a, 31c) are:

$$\nu_{dd,S} = \nu_{S,\max} \left(1 - \frac{R_\rho - 1}{R_{\rho 0} - 1} \right)^3 \quad (37)$$

$$\nu_{dd,T} = 0.7 \nu_{dd,S} \quad (38)$$

11.3 Diffusive Convection ($\alpha dT < 0, \beta dS < 0$)

This occurs when cold fresh water overlies warm salty water (typical of polar regions). The diffusivities (LMD94, Eqs. 32, 34) are:

$$\nu_{dd,T} = \nu_{\text{mol}} \cdot 0.909 \exp \left[4.6 \exp \left(-5.4 \left(\frac{1}{R_\rho} - 1 \right) \right) \right] \quad (39)$$

$$\nu_{dd,S} = \nu_{dd,T} \cdot \max(0.15 R_\rho, 1.85 R_\rho - 0.85) \quad (40)$$

```

1  c      a. Salt fingering
2      IF (      alphaDT(i,j) .GT. betaDS(i,j)
3      &      .AND. betaDS(i,j) .GT. 0. _d 0 ) THEN
4      Rrho = MIN( alphaDT(i,j)/betaDS(i,j), Rrho0 )
5      nutmp      = ( 1. _d 0 - (Rrho - 1. _d 0) * rFac )
6      nudds(i,j) = dsfmax * nutmp * nutmp * nutmp      ! Eq. 31a
7      nuddt(i,j) = 0.7 _d 0 * nudds(i,j)              ! Eq. 31c
8
9  c      b. Diffusive convection
10     ELSEIF ( alphaDT(i,j) .LT. 0. _d 0
11     &      .AND. betaDS(i,j) .LT. 0. _d 0
12     &      .AND. alphaDT(i,j) .GT. betaDS(i,j) ) THEN
13     Rrho = alphaDT(i,j)/betaDS(i,j)
14     nuddt(i,j) = numol * 0.909 _d 0
15     &      * exp ( 4.6 _d 0 * exp (
16     &      - 5.4 _d 0 * ( 1. _d 0/Rrho - 1. _d 0 ) ) )      ! Eq. 32
17     nudds(i,j) = nuddt(i,j) * MAX( 0.15 _d 0 * Rrho,
18     &      1.85 _d 0 * Rrho - 0.85 _d 0 )              ! Eq. 34
19     ENDIF

```

Listing 22: `kpp_routines.F` (KPP_DOUBLEDIFF) – Salt fingering and diffusive convection contributions.

11.4 Shortwave and Salt Plume Fraction Diagnostics

After KPPMIX returns, KPP_CALC computes two fractional penetration fields.

11.5 KPPfrac – Shortwave Penetration Fraction

This is the fraction of shortwave radiation that *does not* contribute to heating within the mixed layer (i.e., passes through it):

$$\text{KPPfrac} = 1 - I(h_{\text{bl}}) \quad (41)$$

where $I(h_{\text{bl}})$ is the fraction of surface shortwave radiation that has been absorbed above depth h_{bl} , computed via SWFRAC.

11.6 KPPplumefrac – Salt Plume Penetration Fraction

Similarly, for the salt plume parameterization:

$$\text{KPPplumefrac} = 1 - P(h_{\text{bl}}) \quad (42)$$

where $P(h_{\text{bl}})$ is the fraction of brine rejection flux that has been distributed above h_{bl} .

12 Eddy Coefficients and Model Interface

These routines are called *every model timestep* by the tracer and momentum solvers, independently of the KPP computation frequency. They simply retrieve the pre-computed KPP fields.

12.1 KPP_CALC_DIFF_T and KPP_CALC_DIFF_S

Both routines have identical structure. They copy the appropriate KPP diffusivity array into the calling routine's work array:

```

1      IF ( kArg .EQ. 0 ) THEN
2  C-    do all levels :
3          DO k=1,MIN(Nr,kSize)
4              DO j=jMin,jMax
5                  DO i=iMin,iMax
6                      KappaRT(i,j,k) = KPPdiffKzT(i,j,k,bi,bj)
7                  ENDDO
8              ENDDO
9          ENDDO
10     ELSE
11  C-    do level k=kArg only :
12          k = MIN(kArg,kSize)
13          DO j=jMin,jMax
14              DO i=iMin,iMax
15                  KappaRT(i,j,k) = KPPdiffKzT(i,j,kArg,bi,bj)
16          ENDDO

```

```

17      ENDDO
18      ENDIF

```

Listing 23: `kpp_calc_diff_t.F` – Retrieval of temperature diffusivity for the tracer solver.

The `kArg` argument allows the calling code to either request a single level ($k > 0$) or all levels ($k = 0$), providing flexibility for the main model’s tracer diffusion loop.

12.2 KPP_CALC_DIFF_PTR

For passive tracers, the diffusivity is `KPPdiffKzS` (salt diffusivity), but with a tracer-specific correction to the background:

```

1      KappaRx(i,j,k) = KPPdiffKzS(i,j,k,bi,bj)
2  #ifndef ALLOW_3D_DIFFKR
3      &          - diffKrNrS(k) + PTRACERS_diffKrNr(k,iTr)
4  #endif

```

Listing 24: `kpp_calc_diff_ptr.F` – Passive tracer diffusivity with tracer-specific background correction.

This correction replaces the default background salt diffusivity (`diffKrNrS`) with the tracer-specific background diffusivity (`PTRACERS_diffKrNr`), while keeping the KPP-computed increment unchanged.

12.3 KPP_CALC_VISC

Adds the KPP viscosity contribution to the momentum viscosity arrays. The KPP viscosity (defined on the t-grid) is averaged to the u-grid and v-grid:

```

1      DO j=jMin,jMax
2          DO i=iMin,iMax
3              KappaRU(i,j,k) = max ( KappaRU(i,j,k),
4              &          KappaRU(i,j,k) - viscArNr(1) + _maskW(i,j,k,bi,bj) *
5              &          0.5*(KPPviscAz(i,j,k,bi,bj)
6              &          +KPPviscAz(i-1,j,k,bi,bj)) )
7          ENDDO
8      ENDDO

```

Listing 25: `kpp_calc_visc.F` – KPP viscosity applied to momentum solver work arrays.

The construction `KappaRU - viscArNr + 0.5*(...)` replaces the background viscosity component with the KPP viscosity, interpolated to the u-point by averaging over the two adjacent t-points. The `max` ensures the result is never less than the background value.

12.4 KPP_TRANSPORT_T

Applies the non-local heat flux term. The effective surface heat flux used for the non-local parameterization must account for the fraction of shortwave radiation that penetrates below the OBL:

$$\langle w'T' \rangle_s = Q_T + Q_{\text{ice}} - Q_{\text{sw}} \frac{\text{KPPfrac}}{C_p \rho_0} \quad (43)$$

```

1      DO j=jMin,jMax
2          DO i=iMin,iMax
3              df(i,j) = - _rA(i,j,bi,bj)
4              &          *KPPdiffKzT(i,j,k,bi,bj)
5              &          *KPPghat(i,j,km1,bi,bj)
6              &          *( surfaceForcingT(i,j,bi,bj)
7              &          +adjustColdSST_diag(i,j,bi,bj)
8              &          -Qsw(i,j,bi,bj)*recip_Cp*recip_rhoConst
9              &          *KPPfrac(i,j,bi,bj)
10             &          )
11         ENDDO
12     ENDDO

```

Listing 26: kpp_transport_t.F – Non-local temperature flux including shortwave correction.

Indexing convention: KPPdiffKzT(k) is defined at the *top* of grid cell k , while KPPghat(km1) is defined at the *bottom* of grid cell $k - 1$ (equivalently, at the top of cell k). The convention ensures:

- At $k = 1$: KPPdiffKzT(1) = 0, so there is no flux through the sea surface.
- KPPghat uses index km1 = k-1 to get the non-local coefficient at the interface above cell k .

12.5 KPP_TRANSPORT_S

Analogous to KPP_TRANSPORT_T for salinity. When ALLOW_SALT_PLUME is active, the surface salt flux is augmented by the fraction of brine rejection that has not yet penetrated below the mixed layer:

```

1      df(i,j) = - _rA(i,j,bi,bj)
2      &          *KPPdiffKzS(i,j,k,bi,bj)
3      &          *KPPghat(i,j,km1,bi,bj)
4      &          *( surfaceForcingS(i,j,bi,bj)
5      #ifdef ALLOW_SALT_PLUME
6      &          + tmpFac1*saltPlumeFlux(i,j,bi,bj)*recip_rhoConst
7      &          *(1.-KPPplumefrac(i,j,bi,bj))
8      #endif
9      &          )

```

Listing 27: kpp_transport_s.F – Non-local salinity flux with optional salt plume contribution.

13 Initialization and Restart

13.1 KPP is Diagnostic, Not Prognostic

A critical characteristic of the KPP scheme is that it is **diagnostic** rather than **prognostic**. This distinction has important implications for initialization, restart procedures, and model behavior.

13.1.1 What “Diagnostic” Means

In KPP, all mixing coefficients—vertical eddy viscosity (`KPPviscAz`), temperature diffusivity (`KPPdiffKzT`), and salinity diffusivity (`KPPdiffKzS`)—are *recomputed from scratch* at each time step based solely on the *instantaneous ocean state*:

- **Current velocity field** (`UVEL`, `VVEL`) determines shear
- **Current temperature and salinity** (`THETA`, `SALT`) determine stratification
- **Current surface forcing** (wind stress, heat flux, freshwater flux) determines boundary layer depth and turbulent velocity scales

There is *no memory* of previous mixing coefficients or turbulent quantities. Each call to `KPP_CALC` is independent of the previous call, except insofar as the ocean state itself has evolved due to the mixing applied in previous time steps.

13.1.2 No State Persistence Between Time Steps

Unlike prognostic turbulence schemes (such as GGL90, which solves a transport equation for turbulent kinetic energy), KPP has **no prognostic state variables**. Specifically:

- KPP does not carry any turbulence-related field (e.g., TKE, dissipation rate, mixing length) from one time step to the next
- The boundary layer depth h_{bl} is *diagnosed* each time step from the bulk Richardson number criterion, not predicted by a temporal evolution equation
- The turbulent velocity scales w_m and w_s are looked up from pre-computed tables based on instantaneous forcing, not integrated forward in time

13.1.3 No Pickup/Restart Files for KPP

Because KPP maintains no prognostic state, **KPP requires no pickup or restart files**. When restarting a MITgcm simulation:

Restart Procedure:

1. Read ocean state pickup files: `pickup.*.data` (contains `THETA`, `SALT`, `UVEL`, `VVEL`, sea surface height, etc.)
2. **KPP arrays are recomputed** immediately on the first time step after restart by calling `KPP_CALC`

3. No KPP-specific pickup file is needed or written

This contrasts sharply with GGL90, which writes a `pickup_ggl90.*.data` file containing the prognostic TKE field at every checkpoint. Without this pickup file, a GGL90 simulation cannot be restarted accurately because the TKE history is lost.

13.2 Initialization

KPP initialization occurs in three stages, as described in detail in Section ??:

1. `KPP_READPARMS`: Read runtime parameters from `data.kpp`
2. `KPP_INIT_FIXED`: Build turbulent velocity scale lookup tables and compute derived constants (one-time, independent of ocean state)
3. `KPP_INIT_VARIA`: Initialize KPP diagnostic arrays (`KPPhbl`, `KPPviscAz`, etc.) to default background values

Critically, `KPP_INIT_VARIA` does *not* set meaningful mixing coefficients—it merely initializes arrays to avoid undefined values. The *first call* to `KPP_CALC` at $t = 0$ computes physically consistent mixing coefficients based on the initial ocean state.

13.3 Implications of Diagnostic Formulation

13.3.1 Advantages

- **Rapid initialization**: No need to spin up turbulent quantities; ocean simulations can start immediately from rest or from observed profiles
- **No pickup file complexity**: Simplifies restart logistics and reduces I/O burden
- **Instantaneous response to forcing**: Changes in wind stress or surface heat flux immediately affect mixing (within one time step), which can be desirable for process studies
- **No spurious transients**: Restarting a simulation produces bit-identical results (modulo floating-point order differences) because there is no turbulent state to reconstruct

13.3.2 Limitations

- **No memory of past forcing**: Turbulent mixing depends only on the instantaneous forcing, not on the integrated history of wind and buoyancy forcing. This can under-represent the persistence of boundary layer turbulence after a wind event
- **Sharp threshold behavior**: The boundary layer depth diagnosis from bulk Richardson number can lead to discontinuous changes in h_{bl} when stratification changes abruptly (e.g., during restratification events)
- **Less suitable for transient phenomena**: Short-timescale turbulent processes (e.g., spin-up/spin-down of TKE during diurnal cycles) are not explicitly resolved

13.4 Contrast with GGL90 (Prognostic TKE Scheme)

For reference, Table 5 compares restart behavior between KPP and GGL90.

Table 5: Initialization and restart: KPP vs. GGL90.

Aspect	KPP (Diagnostic)	GGL90 (Prognostic)
Prognostic variables	None	TKE(x, y, z, t)
Pickup file needed?	No	Yes (<code>pickup_ggl90.*.data</code>)
Restart without pickup	Exact continuation	Incorrect (cold start)
Initialization spin-up	Not needed	TKE equilibrates over ~ 10 days
Memory of past forcing	None	Integrated through TKE PDE
Transient response	Instantaneous	Gradual (governed by TKE timescales)
Boundary layer depth	Diagnosed from Ri_{cr}	Implicit in TKE decay scale
Computational cost	Lower	Higher (implicit PDE solve)

13.5 Practical Restart Workflow

When restarting a KPP-based MITgcm simulation, the following files are sufficient:

```

1 pickup.0000036000.data      # Ocean state (T, S, U, V, eta, etc
   .)
2 pickup.0000036000.meta      # Metadata for ocean state
3 pickup_seaice.0000036000.*  # If using pkg/seaice
4 pickup_obcs.0000036000.*    # If using pkg/obcs
5 # NO pickup_kpp.*.data needed

```

Listing 28: Typical restart file list for KPP simulation.

Upon restart, KPP_CALC is called on the first time step and recomputes all mixing coefficients. The simulation continues exactly as if it had not been interrupted, with no loss of accuracy.

13.6 Recommendations

- For **long climate integrations**, consider using a prognostic scheme (GGL90 or similar) to capture turbulent memory effects
- For **process studies** or **rapid initialization scenarios** (data assimilation, idealized experiments), KPP's diagnostic formulation is advantageous
- If using KPP in a state estimation context, be aware that the lack of turbulent state means mixing responds instantaneously to control adjustments, which can accelerate convergence but may also introduce unphysical responsiveness

14 Compile-Time Options

The compile-time behavior of the KPP package is controlled by CPP macros in `KPP_OPTIONS.h`. Table 6 summarizes the most important flags.

Flag	Default	Effect
<code>KPP_SMOOTH_SHSQ</code>	<code>#define</code>	Apply 121 horizontal filter to $ \delta U ^2$ before Ri computation; reduces noise in shear-driven mixing
<code>KPP_SMOOTH_DBLOC</code>	<code>#define</code>	Apply 121 horizontal filter to δb_{loc} before Ri computation
<code>KPP_SMOOTH_DENS</code>	<code>#undef</code>	Apply 121 filter to all density-related fields; implies <code>KPP_SMOOTH_DBLOC</code>
<code>KPP_SMOOTH_VISC</code>	<code>#undef</code>	Apply 121 horizontal filter to <code>KPPviscAz</code>
<code>KPP_SMOOTH_DIFF</code>	<code>#undef</code>	Apply 121 horizontal filter to <code>KPPdiffKzS/T</code>
<code>KPP_GHAT</code>	<code>#define</code>	Include non-local transport term $\hat{\gamma}$ in scalar fluxes
<code>EXCLUDE_KPP_SHEAR_MIX</code>	<code>#undef</code>	Disable shear instability mixing in <code>Ri_iwmix</code>
<code>EXCLUDE_KPP_DOUBLEDIFF</code>	<code>#undef</code>	Disable double-diffusion parameterization
<code>KPP_ESTIMATE_UREF</code>	<code>#undef</code>	Use resolution-independent surface reference velocity for <code>dVsq</code>
<code>KPP_DO_NOT_MATCH_DIFFUSIVITY</code>	<code>#undef</code>	Skip OBL/interior matching in <code>blmix</code> ; can reduce noise at OBL base
<code>KPP_SCALE_SHEARMIXING</code>	<code>#undef</code>	Scale shear mixing by $ \delta U ^4 / (\delta U ^4 + 10^{-16})$ following Polzin (1996)
<code>ALLOW_KPP_VERTICALLY_SMOOTH</code>	<code>#undef</code>	Enable vertical 121 smoothing of Richardson number
<code>KPP_SMOOTH_REGULARISATION</code>	<code>#undef</code>	Replace <code>MAX(x,eps)</code> with <code>x+eps</code> for smoother adjoint behavior

Table 6: KPP compile-time CPP options from `KPP_OPTIONS.h`.

Overlap requirement: Any flag that activates horizontal smoothing (KPP_SMOOTH_SHSQ, KPP_SMOOTH_DBLOC, KPP_SMOOTH_DENS) requires the model overlap region to be at least 4 halo cells (OLx=OLy=4). This is checked at startup by KPP_CHECK. The smoothing operations in KPP_CALC use stencils that extend to jm1/jp1 relative to the jMin/jMax computation range.

15 Output and Diagnostics

15.1 State File Output (KPP_OUTPUT)

When `KPPwriteState = .TRUE.`, the following fields are written to binary files at intervals of `kpp_dumpFreq`:

File prefix	Content
KPPviscAz.	Vertical eddy viscosity
KPPdiffKzT.	Temperature diffusivity
KPPdiffKzS.	Salt diffusivity
KPPghat.	Non-local coefficient
KPPhbl.	Boundary layer depth

15.2 Online Diagnostics (KPP_DIAGNOSTICS_INIT)

The following diagnostics are registered with the MITgcm diagnostics package and can be requested via `data.diagnostics`:

Diagnostic Name	Dims	Description	Units
KPPviscA	3D	Vertical eddy viscosity	m ² /s
KPPdiffS	3D	Salt/tracer diffusivity	m ² /s
KPPdiffT	3D	Heat diffusivity	m ² /s
KPPghatK	3D	Non-local flux ratio	0–1
KPPhbl	2D	Boundary layer depth	m
KPPfrac	2D	SW fraction in OBL	–
KPPdbsfC	3D	Buoyancy diff. vs surface	m/s ²
KPPbfsfC	3D	Eff. surface buoyancy forcing	m ² /s ³
KPPRi	3D	Bulk Richardson number	–
KPPbo	2D	Turbulent buoyancy forcing	m ² /s ³
KPPbosol	2D	Radiative buoyancy forcing	m ² /s ³
KPPdbloc	3D	Local buoyancy gradient	m/s ²
KPPshsq	3D	Local shear squared	m ² /s ²
KPPdVsQ	3D	Surface-ref. shear squared	m ² /s ²
KPPnuddt	3D	Double-diff. heat diffusivity	m ² /s
KPPnudds	3D	Double-diff. salt diffusivity	m ² /s

Diagnostic Name	Dims	Description	Units
KPPgTH01	3D	Non-local temperature flux	$^{\circ}\text{C}\cdot\text{m}^3/\text{s}$
KPPgSLT1	3D	Non-local salinity flux	$\text{g}/\text{kg}\cdot\text{m}^3/\text{s}$
KPPpfrac	2D	Salt plume OBL fraction	—
KPPbopl _m	2D/3D	Haline buoyancy forcing	m^2/s^3

Table 7: Available KPP diagnostics registered by KPP_DIAGNOSTICS_INIT.

16 Package Validation: KPP_CHECK

KPP_CHECK validates the model configuration at startup. It enforces the following requirements:

Requirement	Reason
<code>implicitDiffusion = .TRUE.</code>	KPP diffusivities must be applied implicitly
<code>implicitViscosity = .TRUE.</code>	KPP viscosity must be applied implicitly
<code>cAdjFreq = 0</code>	KPP handles all convection internally
<code>ivdc_kappa = 0</code>	No convective adjustment allowed with KPP
<code>OLx = 0</code> , <code>OLy = 4</code>	Required when horizontal smoothing is enabled

Convection handling: KPP handles static instability via the f_{con} function in `Ri_iwmix`, which sets very large diffusivities (`difmcon`, `difscon`, `diftcon` $\sim 0.1 \text{ m}^2/\text{s}$) when $N^2 < 0$. Therefore, any other convective parameterization (`convAdjFreq`, `ivdc_kappa`) must be disabled to avoid double-counting.

17 Numerical Considerations & Frequently Encountered Issues

- Negative diffusivities in shallow regions:** In 2-level columns or when the mixed layer is very shallow (less than one grid cell), `blmix` can return negative values for `blmc`. These are guarded against by the `MAX` operators in the final combination step of `KPPMIX` (Listing 21).
- KPP computation frequency vs. timestep:** Setting `kpp_freq` $\gg$ `deltaTClock` is a common efficiency optimization but can cause subtle inconsistencies: the KPP fields are applied at every model timestep using stale forcing data. This is generally acceptable for large-scale simulations but may cause issues in rapidly evolving systems (e.g., storm passages).
- Indexing of `vddiff`:** The internal array `vddiff(i,j,k,mdiff)` uses index `k-1` for the interface above level `k`. Hence `vddiff(:, :, 0, :)` is the surface interface (always

zero) and `vddiff(:, :, Nr, :)` is the bottom interface. The transfer loop uses `k-1` to fill `KPPviscAz/diffKzT/S(k)`.

4. **Shelfice compatibility:** When `useShelfIce = .TRUE.`, the surface index `kSurf` is set to the top of the cavity, and boundary layer mixing coefficients are reset to interior values under the shelf (where `msk(i) = 0`).
5. **Automatic differentiation (TAF/TAPENADE):** Numerous `CADJ STORE` directives appear throughout `kpp_routines.F` and `kpp_calc.F`. These store checkpoints on the “comlev1_kpp” tape for the adjoint computation. The flag `KPP_AUTODIFF_EXCESSIVE_STORE` can be defined to store additional variables at the cost of memory, which may be needed for very long adjoint runs.

18 Conclusions

The MITgcm KPP package provides a complete, self-consistent implementation of the Large-McWilliams-Doney (1994) K-Profile Parameterization. The package:

- Separates the water column into a turbulent boundary layer (above h_{bl}) and a stratified interior (below h_{bl})
- Derives h_{bl} from a bulk Richardson number criterion that accounts for both resolved velocity shear and unresolved turbulent shear
- Uses a smooth, physically-motivated cubic polynomial profile for mixing within the OBL that exactly matches interior diffusivities at the OBL base
- Includes a non-local counter-gradient transport term that is active only under unstable surface forcing
- Computes interior mixing from the local gradient Richardson number plus background internal-wave contributions
- Optionally includes double-diffusion from salt fingering and diffusive convection
- Interfaces cleanly with the MITgcm tracer and momentum solvers through dedicated retrieval routines
- Supports offline usage, automatic differentiation for adjoint modeling, and a full suite of online diagnostics

The modular structure – with the surface forcing, boundary layer depth search, boundary layer mixing, and interior mixing all in separate subroutines – makes the code relatively straightforward to modify, e.g., to implement alternative OBL depth criteria or different interior mixing parameterizations.

19 Glossary of Symbols

Symbol	Code name	Definition
h_{bl}	hbl, KPPhbl	Oceanic planetary boundary layer depth (m)
k_{bl}	kbl	Model level index of first cell below h_{bl}
u_*	ustar	Surface friction velocity (m/s)
B_0	bo	Surface turbulent buoyancy flux (m^2/s^3)
$B_{0,sol}$	bosol	Surface radiative buoyancy flux (m^2/s^3)
B_f	bfsfc	Total surface buoyancy flux at $z = -h_{bl}$ (m^2/s^3)
w_m	wm	Turbulent velocity scale for momentum (m/s)
w_s	ws	Turbulent velocity scale for scalars (m/s)
σ	sigma	Normalized depth d/h_{bl}
$\hat{\zeta}$	zemat	$\kappa \sigma h_{bl} B_f$ (m^3/s^3)
Ri_b	Rib	Bulk Richardson number
Ri_{cr}	Ricr	Critical bulk Richardson number (= 0.3)
Ri_{loc}	local Ri in Ri_iwmix	Local gradient Richardson number
Ri_∞	Riinfty	Ri limit for shear instability (= 0.7)
N^2	bvsq	Brunt-Väisälä frequency squared (s^{-2})
f_{con}	fcon	Convective mixing function
f_{Ri}	fRi	Shear instability mixing function
$G(\sigma)$	Gm, Gs, Gt	Shape function in boundary layer
$\hat{\gamma}$	ghat, KPPghat	Non-local transport coefficient (s/m^2)
R_ρ	Rrho	Density ratio for double diffusion
α	TTALPHA	Thermal expansion $d\rho/dT$ ($kg/m^3/^\circ C$)
β	SSBETA	Haline contraction $d\rho/dS$ ($kg/m^3/(g/kg)$)
κ	vonk	Von Kármán constant (= 0.4)
ϵ	epsilon	Nondimensional surface layer fraction (= 0.1)

A Complete Input/Output Mapping

A.1 Package-Level Inputs

Input	Source	Description	Units
uVel, vVel	DYNVARS	Horizontal velocity	m/s
theta	DYNVARS	Potential temperature	°C
salt	DYNVARS	Salinity	g/kg
surfaceForcingU/V	FFIELDS	Wind stress / ρ	m ² /s ²
surfaceForcingT	FFIELDS	Net heat flux forcing	K·m/s
surfaceForcingS	FFIELDS	Net salt flux forcing	(g/kg)·m/s
adjustColdSST_diag	FFIELDS	Sea-ice heat flux correction	K·m/s
Qsw	FFIELDS	Shortwave radiation	W/m ²
fCori	GRID	Coriolis parameter	1/s
maskC	GRID	Land/ocean mask at t-points	–
kLowC	GRID	Index of deepest wet level	–
rC, drF	GRID	Vertical grid	m
viscArNr	PARAMS	Background vertical viscosity	m ² /s
saltPlumeFlux	SALT_PLUME	Brine rejection flux	kg/m ² /s
SaltPlumeDepth	SALT_PLUME	Salt plume penetration depth	m
SWFrac3D	FFIELDS	3D SW attenuation profile	–

A.2 Package-Level Outputs

Output	Consumed by	Description	Units
KPPviscAz	KPP_CALC_VISC → CALC_DIFFUSIVITY	Vertical eddy viscosity	m ² /s
KPPdiffKzT	KPP_CALC_DIFF_T → GAD_DIFF	Temperature diffusivity	m ² /s
KPPdiffKzS	KPP_CALC_DIFF_S/SALT → GAD_DIFF	Salt/tracer diffusivity	m ² /s
KPPghat	KPP_TRANSPORT_T/NOTA	Vertical local coefficient	s/m ²
KPPhbl	KPP_OUTPUT, diagnostics	Boundary layer depth	m
KPPfrac	KPP_TRANSPORT_T	SW fraction penetrating OBL	–
KPPplumefrac	KPP_TRANSPORT_S	Salt plume fraction	–
KappaRT	CALC_DIFFUSIVITY	Temperature diffusivity work array	m ² /s
KappaRS	CALC_DIFFUSIVITY	Salinity diffusivity work array	m ² /s

Output	Consumed by	Description	Units
KappaRU, KappaRV	CALC_DIFFUSIVITY	Momentum viscosity work arrays	m^2/s
df (T,S,PTR)	GAD_DIFF family	Non-local tracer flux	tracer· m^3/s

B Adjoint Model Considerations

The KPP package supports adjoint-based optimization through MITgcm’s automatic differentiation (AD) framework, making it suitable for gradient-based data assimilation, parameter estimation, and sensitivity studies. This appendix documents adjoint-specific features, configuration options, and best practices for using KPP in adjoint mode.

B.1 Overview: KPP in Adjoint Mode

MITgcm’s automatic differentiation tool (primarily TAF, Transformation of Algorithms in Fortran) generates adjoint code from the forward model source. The adjoint model propagates sensitivities backward in time, computing gradients of a cost function (e.g., misfit to observations) with respect to control parameters (e.g., initial conditions, surface forcing, mixing parameters).

B.1.1 KPP’s Adjoint Characteristics

KPP’s diagnostic formulation presents both advantages and challenges for adjoint applications compared to prognostic schemes like GGL90:

Advantages:

- **Lower memory requirements:** No prognostic TKE field to checkpoint, reducing tape storage needs

- **Simpler forward computation:** Diagnostic calculation is faster than implicit PDE solve, reducing adjoint recomputation cost
- **No spin-up artifacts:** Instantaneous response to forcing changes means adjoint sensitivities propagate directly without turbulent memory

Challenges:

- **Many conditional branches:** Boundary layer depth search (`bldepth`) and profile shape function matching (`enhance`) involve `IF` statements that create gradient discontinuities

- **Lookup table interpolation:** Turbulent velocity scales (w_m, w_s) are computed via bilinear interpolation from pre-tabulated functions, which TAF must differentiate carefully
- **Non-smooth dependencies:** The bulk Richardson number criterion ($\text{Ri}_b = \text{Ri}_{\text{cr}}$) produces a sharp threshold in h_{bl} , leading to potentially noisy adjoint gradients near the transition

B.2 Adjoint-Specific Implementation

B.2.1 TAF Directives in KPP Source Code

The KPP source files contain extensive `CADJ` directives that guide TAF’s code generation. These directives fall into three categories:

1. Checkpointing Directives (`CADJ STORE`) Intermediate variables that are recomputed during the adjoint backward sweep must be stored (checkpointed) on “tapes.” The primary KPP tape is `comlev1_kpp`, indexed by `ikey` (depends on tile indices).

Example from `kpp_routines.F`:

```

1 CADJ STORE ghat = comlev1_kpp, key=ikey, kind=isbyte
2 CADJ STORE dbloc = comlev1_kpp, key=ikey, kind=isbyte
3 ...
4 CADJ STORE hbl,bfsfc,stable,casea,kbl = comlev1_kpp,
5 CADJ & key=ikey, kind=isbyte

```

Listing 29: TAF checkpoint directives in KPPMIX.

Key variables stored on tape include:

- `hbl`: Boundary layer depth (controls profile shape)
- `ghat`: Non-local transport coefficient (used in tracer fluxes)
- `dbloc`: Local buoyancy gradient (affects Richardson number)
- `bfsfc`: Surface buoyancy forcing at $z = -h_{bl}$ (affects turbulent velocity scales)
- `stable`: Stability flag (determines lookup table branch)
- `kbl`: Grid index of boundary layer base (affects stencil loops)

2. Incomplete Variable Declarations (`CADJ INCOMPLETE`) Some intermediate work arrays are declared `INCOMPLETE` to suppress TAF warnings about uninitialized variables. This is safe when the array is fully populated before use.

Example from `kpp_routines.F`:

```

1 CADJ INCOMPLETE worka
2 CADJ INCOMPLETE blmc

```

Usage note: `worka` is a temporary array for shortwave penetration and salt plume calculations; `blmc` holds boundary layer mixing coefficients computed by `blmix`.

3. Vertical-Level-Dependent Keys When looping over vertical levels within a tile, TAF requires a finer-grained tape key to handle dependencies correctly:

```

1  C      kkey :: tape key TAF-AD simulations (depends on vertical levels
      and tiles)
2      INTEGER kkey
3      ...
4      kkey = (ikey-1)*Nr + k
5  CADI STORE worka = comlev1_kpp_k, key = kkey, kind=isbyte

```

This pattern appears in KPPMIX, Ri_iwmix, and blmix where vertical loops have inter-level dependencies.

B.2.2 Adjoint Differentiation List

The file `kpp_ad_diff.list` specifies which KPP routines are differentiated by TAF. Contents:

```

1  kpp_calc.f
2  kpp_calc_diff_t.f
3  kpp_calc_diff_s.f
4  kpp_calc_diff_ptr.f
5  kpp_calc_visc.f
6  kpp_output.f
7  kpp_do_exch.f
8  kpp_init_varia.f
9  kpp_routines.f
10 kpp_transport_s.f
11 kpp_transport_t.f
12 kpp_transport_ptr.f
13 kpp_forcing_surf.f

```

Listing 30: `pkg/kpp/kpp_ad_diff.list`

Notably *excluded* from differentiation:

- `kpp_readparms.f`: Parameter initialization (no state dependence)
- `kpp_init_fixed.f`: Lookup table generation (one-time setup)
- `kpp_check.f`: Validation checks (diagnostic only)
- `kpp_diagnostics_init.f`: Diagnostics registration (no state)

B.3 Adjoint-Specific CPP Flags

B.3.1 KPP_AUTODIFF_EXCESSIVE_STORE

Location: `KPP_OPTIONS.h`, line 72

Purpose: Store additional intermediate variables on tape to avoid extensive recomputations during the adjoint backward sweep.

Default: `#undef` (minimal checkpointing)

Effect when defined: Increases memory usage but reduces adjoint runtime by avoiding recomputation of:

- Intermediate buoyancy gradients in `STATEKPP`
- Shape function coefficients in `blmix`
- Richardson number profiles in `Ri_iwmix`

Trade-offs:

- **Pro:** Faster adjoint execution ($\sim 20\text{--}30\%$ speedup)
- **Con:** Larger checkpoint files ($\sim 1.5\text{--}2\times$ memory)
- **Pro:** Reduces cumulative floating-point error from recomputation

Recommendation: Enable for production adjoint runs with sufficient memory. Leave undefined for testing and memory-constrained systems.

Code reference: `kpp_routines.F` (multiple `#ifdef KPP_AUTODIFF_EXCESSIVE_STORE` blocks around `CADJ STORE` directives)

B.4 Known Adjoint Issues and Workarounds

B.4.1 Issue 1: Gradient Noise at Boundary Layer Depth Transitions

Symptom: Adjoint gradients exhibit noise (large oscillations) when h_{bl} transitions between grid levels, particularly in regions with marginal stratification (bulk Richardson number near $Ri_{cr} = 0.3$).

Cause: The linear interpolation in `bldepth` to find h_{bl} is piecewise linear but not smooth. When Ri_b crosses Ri_{cr} , the derivative $\partial h_{bl} / \partial \text{forcing}$ can be large or discontinuous.

Workarounds:

- **Increase `minKPPhbl`:** Setting a larger minimum boundary layer depth (e.g., `minKPPhbl` = 5.0 m instead of 1.0 m) reduces shallow-layer sensitivity
- **Use `KPP_SMOOTH_REGULARISATION`:** When defined in `KPP_OPTIONS.h`, this replaces some `MAX(var, epsilon)` with `var + epsilon`, providing smoother gradients (but changes forward results slightly)
- **Horizontal smoothing:** Enable `KPP_SMOOTH_DBLOC` and `KPP_SMOOTH_SHSQ` to reduce grid-scale noise in buoyancy and shear fields that feed into Ri_b
- **Cost function design:** Avoid cost terms that depend directly on h_{bl} (e.g., “boundary layer depth matches observed MLD”); instead use integrated quantities like heat content or temperature profile misfits

B.4.2 Issue 2: Conditional Branch Discontinuities

Symptom: Finite-difference gradient checks fail at specific locations (usually related to IF statements in `enhance`, `blmix`, or stability flag logic).

Cause: TAF generates adjoint code for both branches of an IF, but the choice of branch depends on state variables. Small perturbations can flip branches, causing gradient discontinuities.

Workarounds:

- **Accept finite-difference mismatch:** In practice, optimization algorithms (L-BFGS, conjugate gradient) are robust to small gradient errors
- **Use smooth activation functions:** If modifying KPP source, replace sharp IF (condition) with smooth transitions (e.g., sigmoid or hyperbolic tangent), but this changes physics
- **Stabilize stratification:** Ensure model resolution and time stepping are adequate to avoid rapid stratification changes that trigger branch flips

B.4.3 Issue 3: Lookup Table Interpolation Artifacts

Symptom: Adjoint sensitivities to surface forcing (u_* , B_0) show structured patterns corresponding to lookup table grid spacing.

Cause: Turbulent velocity scales w_m and w_s are computed via bilinear interpolation in `wscale` from pre-tabulated 2D arrays (functions of ζ and σ). TAF differentiates the interpolation stencil, which can amplify discretization errors.

Workarounds:

- **Increase lookup table resolution:** Recompile with finer ζ spacing in `kpp_init_fixed.F` (requires modifying hardcoded table generation)
- **Use cubic interpolation:** Modify `wscale` to use cubic splines instead of bilinear (smoother derivatives but higher cost)
- **Limit parameter search range:** Constrain forcing perturbations to stay within well-sampled regions of the lookup table

B.5 KPP vs. GGL90: Adjoint Complexity Comparison

Table 12 compares adjoint characteristics of KPP and GGL90.

Key takeaway: GGL90's prognostic formulation generally provides better gradient quality and faster optimization convergence, but KPP's lower memory footprint and simpler forward calculation can be advantageous in memory-constrained or rapid-turnaround scenarios.

Table 12: Adjoint complexity comparison: KPP vs. GGL90.

Aspect	KPP	GGL90
Primary equation type	Diagnostic (BL depth search)	PDE (TKE transport)
Conditional branches	Many (~ 30 – 50)	Few (~ 5 – 10)
Gradient smoothness	Moderate (discontinuous at h_{bl})	High (smooth PDEs)
Parameter count	~ 15 profile coefficients	3–4 core params
Adjoint memory	Low (no prognostic state)	Moderate (TKE field)
Adjoint runtime	$\sim 1.3\times$ forward	$\sim 1.5\times$ forward
Gradient check pass rate	Moderate (70–90%)	High ($>95\%$)
Optimization convergence	Slower (many iterations)	Fast (few iterations)

B.6 Best Practices for Adjoint Runs with KPP

1. **Start with forward-validated parameters:** Tune KPP in forward mode first to match observations. Only then enable adjoint optimization.
2. **Gradient checking:** Run manual finite-difference gradient checks for key cost function components. Expect agreement to 4–8 significant digits (less than GGL90 due to discontinuities).
3. **Parameter bounds:** Constrain optimization parameters to physical ranges:
 - $0.2 < Ricr < 0.5$
 - $0.5 < Riinfy < 1.0$
 - $0.001 < difm0 < 0.01 \text{ m}^2/\text{s}$
 - $5 < cstar < 15$ (nonlocal transport strength)
4. **Cost function design:** Use integrated/smoothed quantities (e.g., mixed layer heat content, time-averaged SST) rather than instantaneous point values. Avoid cost terms that depend on sharp thresholds.
5. **Smoothing options:** Enable KPP_SMOOTH_DBLOC and KPP_SMOOTH_SHSQ to reduce input noise. Consider KPP_SMOOTH_REGULARISATION if gradient checks reveal discontinuity issues.
6. **Checkpoint frequency:** Balance checkpoint frequency against memory constraints. For KPP, checkpointing every 10–50 timesteps is typical. Enable KPP_AUTODIFF_EXCESSIVE_STORE if memory permits.
7. **Optimization algorithm:** Use quasi-Newton methods (L-BFGS) or conjugate gradient, which are robust to moderate gradient noise. Avoid methods requiring Hessian information (Newton, trust-region) due to second- derivative discontinuities.

B.7 Adjoint Validation Test Cases

When enabling KPP in adjoint mode for a new configuration, perform the following validation tests:

Test 1: Gradient Check (Finite Difference) • Perturb a control variable (e.g., `difm0`) by $\epsilon = 10^{-6}$

- Compute finite-difference gradient: $\frac{J(\text{difm0}+\epsilon)-J(\text{difm0})}{\epsilon}$
- Compare to adjoint gradient: expect agreement to 4–8 digits
- Repeat for multiple control variables and multiple perturbation sizes

Test 2: Adjoint-Forward Consistency • Run forward model, save state at checkpoints

- Run adjoint model backward from $t = T$ to $t = 0$
- Verify adjoint state is consistent with forward state at checkpoints
- Check for NaNs, Infs, or abnormally large values

Test 3: Optimization Convergence • Set up a simple twin experiment: generate synthetic observations from a forward run with known parameters

- Perturb parameters by $\sim 20\%$
- Run optimization to recover true parameters
- Check convergence within 50–100 iterations

B.8 References for Adjoint Modeling

For further background on adjoint methods and their application to ocean mixing parameterizations:

- Marotzke, J., R. Giering, K. Q. Zhang, D. Stammer, C. Hill, and T. Lee (1999). *Construction of the adjoint MIT ocean general circulation model and application to Atlantic heat transport sensitivity*. Journal of Geophysical Research, 104(C12), 29,529–29,547.
- Heimbach, P., C. Hill, and R. Giering (2005). *An efficient exact adjoint of the parallel MIT General Circulation Model, generated via automatic differentiation*. Future Generation Computer Systems, 21(8), 1356–1371.
- Forget, G., J.-M. Campin, P. Heimbach, C. N. Hill, R. M. Ponte, and C. Wunsch (2015). *ECCO version 4: An integrated framework for non-linear inverse modeling and global ocean state estimation*. Geoscientific Model Development, 8, 3071–3104.
- Ferreira, D., J. Marshall, and P. Heimbach (2005). *Estimating eddy stresses by fitting dynamics to observations using a residual-mean ocean circulation model and its adjoint*. Journal of Physical Oceanography, 35(10), 1891–1910.